Chapter 16 solutions

16.1 Groups

1

The invertible one-by-one real matrices are nonzero scalars, whose multiplication commutes. Thus GL_1 is abelian.

In dimension 2, choose

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Both determinants are 1, but

$$AB = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \qquad BA = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}.$$

Hence GL_2 is not abelian.

For every $n \geq 2$, the zero class in $\mathbb{Z}_n$ cannot have a multiplicative inverse, because $[0][b] = [0] \neq [1]$. The full set is not a multiplicative group even when n is prime. For prime p, use $\mathbb{Z}_p^\times = \{[1], \ldots, [p-1]\}$. Every such class is coprime to p, and Bezout supplies its inverse; closure follows from the absence of zero divisors.

2

If e, f are identities, then $ef = f$ using e, and $ef = e$ using f, hence $e = f$. If b, c are inverses of a,

$$b = eb = (ca)b = c(ab) = ce = c.$$

If $ab = ac$, left multiplication by a^{-1} and associativity give $eb = ec$, hence $b = c$.

In the additive group $\mathbb{Z}_{12}$, the first positive multiples of 3 are 3, 6, 9, 0. Its order is 4.

16.2 Homomorphisms

3

If $[k]_{18} = [\ell]_{18}$, then $18 \mid k - \ell$, so $6 \mid k - \ell$, proving independence of representatives. Also

$$\varphi([k] + [\ell]) = [k + \ell]_6 = [k]_6 + [\ell]_6.$$

The kernel is $\{[0]_{18}, [6]_{18}, [12]_{18}\}$. Every $[k]_6$ has preimage $[k]_{18}$, so the image is all $\mathbb{Z}_6$. The map from a kernel coset to its common image is well defined and bijective, giving $\mathbb{Z}_{18}/\ker\varphi \cong \mathbb{Z}_6$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 16 solutions (continued)

16.3 Cosets

4

The four distinct cosets are

$$\{0,4,8\}, \quad \{1,5,9\}, \quad \{2,6,10\}, \quad \{3,7,11\}.$$

They are exactly the residue classes modulo 4 within $\mathbb{Z}_{12}$, hence disjoint and collectively exhaustive. Each has three elements. Lagrange gives index $12/3 = 4$, agreeing with the explicit list.

5

Right-to-left multiplication gives

$$gH = \{(13),(13)(12)\} = \{(13),(123)\},$$
$$Hg = \{(13),(12)(13)\} = \{(13),(132)\}.$$

They differ, so H is nonnormal. Write $h = (12)$, so $gh = (123)$. Both g and gh represent gH. Using g for both factors gives $gg = e \in H$. Using gh gives $(gh)^2 = (132) \notin H$. Thus the resulting cosets differ, refuting representative independence.

6

Assume independence. Since $hH = H$ for every $h \in H$,

$$(hg)H = (hH)(gH) = H(gH) = gH.$$

Thus $hg = gh_1$ for some $h_1 \in H$, so $g^{-1}hg \in H$. This gives $g^{-1}Hg \subseteq H$. Apply the same inclusion to g^{-1}, then conjugate, to obtain the reverse inclusion. Hence H is normal.

Conversely, suppose H normal. Representatives of the same left cosets have forms $a' = ah_1, b' = bh_2$. Normality gives $b^{-1}h_1 b = h_3 \in H$, so

$$a'b' = ah_1bh_2 = abh_3h_2 \in abH.$$

Therefore $(a'b')H = (ab)H$, proving independence for all representatives.

16.4 RSA

7

$n = 55$, $\varphi(n) = 40$. Extended Euclid gives

$$40 = 2(17) + 6, \quad 17 = 2(6) + 5, \quad 6 = 5 + 1,$$
$$1 = 6 - 5 = 6 - (17 - 2 \cdot 6) = 3 \cdot 6 - 17 = 3 \cdot 40 - 7 \cdot 17.$$

Thus $17^{-1} \equiv -7 \equiv 33 \pmod{40}$, and take $d = 33$.

Chapter 16 solutions (continued)

Encryption squares modulo 55 are

$$7^2 \equiv 49, \quad 7^4 \equiv 36, \quad 7^8 \equiv 31, \quad 7^{16} \equiv 26.$$

Hence $7^{17} \equiv 26 \cdot 7 = 182 \equiv 17$, so $C = 17$.

For decryption,

$$17^2 \equiv 14, \quad 17^4 \equiv 31, \quad 17^8 \equiv 26, \quad 17^{16} \equiv 16, \quad 17^{32} \equiv 36.$$

Therefore $17^{33} \equiv 36 \cdot 17 = 612 \equiv 7 \pmod{55}$, recovering the message.

8

Modulo 5, $15^{17} \equiv 0$. Modulo 11, reduce 15 to 4 and its exponent modulo 10. Since

$$4^2 \equiv 5, \quad 4^4 \equiv 3, \quad 4^7 \equiv 3 \cdot 5 \cdot 4 \equiv 5,$$

encryption satisfies $C \equiv 0 \pmod{5}, C \equiv 5 \pmod{11}$. Set $C = 5k$. Then $5k \equiv 5 \pmod{11}$ gives $k \equiv 1$, so the least representative is $C = 5$.

For decryption, $5^{33} \equiv 0 \pmod{5}$, and Fermat is valid modulo 11 because 5 is coprime to 11. Thus $5^{33} \equiv 5^3 \equiv 4 \pmod{11}$. Writing $M' = 5k$, solve $5k \equiv 4 \pmod{11}$. The inverse of 5 is 9 there, so $k \equiv 36 \equiv 3$, giving $M' \equiv 15 \pmod{55}$. Euler's theorem modulo 55 was never applied to the noncoprime message.

16.5 Detection and correction

9

The two codewords have distance 3. A membership detector catches every one- and two-bit error. A nearest-codeword decoder corrects all errors of weight at most $\lfloor (3-1)/2 \rfloor = 1$.

When 111 becomes 100, two errors occurred. Distances are $d(111, 100) = 2$ and $d(000, 100) = 1$. Nearest decoding returns 000, a miscorrection. The received word is not a codeword and would be flagged by membership-only detection, but the correction decoder instead assigns it to another codeword's radius-1 ball. Separate maximal guarantees do not promise both behaviors simultaneously.

16.6 Distances

10

$d(x, y) = 3$, at positions 1, 4, 5. $d(x, z) = 2$, at positions 2, 5. $d(y, z) = 3$, at positions 1, 2, 4. Thus $2 \leq 3 + 3$, and the other inequalities $3 \leq 3 + 2$ also hold.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 16 solutions (continued)

$$V(5,2) = \binom{5}{0} + \binom{5}{1} + \binom{5}{2} = 1 + 5 + 10 = 16.$$

Distance 5 guarantees correction of $\lfloor (5-1)/2 \rfloor = 2$ errors.

16.7 Matrices

11

The integer dot products, reduced modulo 2, give

$$GH^T = \begin{bmatrix} 2 & 2 & 0 \\ 2 & 0 & 2 \\ 0 & 2 & 2 \\ 2 & 2 & 2 \end{bmatrix} \equiv \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \pmod 2.$$

For example, the first entry is $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 = 2$. All zero entries in the reduced matrix are field elements of $\mathbb{F}_2$.

Write $G = [I_4 \mid A]$, where the four rows of A are 110, 101, 011, 111. The displayed H is $[A^T \mid I_3]$. Therefore

$$GH^T = [I_4 \mid A] \begin{bmatrix} A \\ I_3 \end{bmatrix} = A + A = 0$$

over $\mathbb{F}_2$. Message 1010 adds rows 1 and 3, producing 1010101. The parity checks are $1 + 0 + 0 + 1 = 0$, $1 + 1 + 0 + 0 = 0$, and $0 + 1 + 0 + 1 = 0$ modulo 2, so its syndrome is zero.

The seven columns of H are 110, 101, 011, 111, 100, 010, 001. None is zero, excluding weight-1 kernel words. No two agree, excluding weight-2 kernel words. Generator row 1 has weight 3, so minimum nonzero weight, and hence minimum distance, is exactly 3.

16.8 Coset leaders

12

The zero-syndrome coset is $C = \{000, 011, 101, 110\}$, with unique minimum-weight leader 000. The other coset is $100 + C = \{100, 111, 001, 010\}$, all of syndrome 1. Its three possible leaders are 100, 010, 001, each of weight 1. Together the cosets partition all eight words.

One syndrome cannot distinguish the three single-error locations. The code has distance 2, guarantees one-error detection, and cannot uniquely correct every single error. A chosen standard array merely resolves a tie according to an arbitrary specified rule.

Chapter 16 solutions (continued)

16.9 Hamming decoding

13

The difference between received and sent words is a 1 in position 2. Because the codeword's syndrome is zero,

$$rH^T = (1, 0, 1),$$

whose transpose is the second column of H. Flipping position 2 maps 1110101 to 1010101.

A radius-1 ball has eight words. There are $2^4 = 16$ codewords, and minimum distance 3 makes those balls disjoint. Their total size $16 \cdot 8 = 128 = 2^7$ is the whole ambient space, proving perfectness.

16.10 Burnside

14

The identity fixes $3^4 = 81$ colorings. Each of the two quarter turns fixes 3, the half turn fixes $3^2 = 9$, each of the two opposite-vertex reflections fixes $3^3 = 27$, and each opposite-edge reflection fixes $3^2 = 9$. Thus

$$\frac{81 + 2 \cdot 3 + 9 + 2 \cdot 27 + 2 \cdot 9}{8} = \frac{168}{8} = 21.$$

For the denominator, count $F = \{(g, x) : g \cdot x = x\}$. By group elements the count is 168. On any orbit O, summing stabilizer sizes gives $|O|(|D_4|/|O|) = 8$. Hence $|F| = 8$ times the orbit count. The divisor is 8, not the 81 raw colorings.

16.11 Cycle index

15

The rotation cycle types give

$$Z_{C_4} = \frac{x_1^4 + x_2^2 + 2x_4}{4}.$$

Setting every variable to 3 gives $(81 + 9 + 6)/4 = 24$. The full square group adds two opposite-vertex reflections of type $1^2 2$, and two opposite-edge reflections of type 2^2:

$$Z_{D_4} = \frac{x_1^4 + x_2^2 + 2x_4 + 2x_1^2 x_2 + 2x_2^2}{8}.$$

Evaluation at 3 gives $(81 + 9 + 6 + 54 + 18)/8 = 21$.

Chapter 16 solutions (continued)

16.12 Polya

16

From $Z_{C_3} = (x_1^3 + 2x_3)/3$, substitute $x_k = r^k + w^k + b^k$. The full inventory is

$$\frac{(r+w+b)^3 + 2(r^3+w^3+b^3)}{3} = r^3 + w^3 + b^3 + r^2w + r^2b + w^2r + w^2b + b^2r + b^2w + 2rwb.$$

The two rwb orbits have cyclic orders red-white-blue and red-blue-white. Rotations preserve these orders; reflection would identify them, but is not in C_3. Substituting $r = w = b = 1$ gives $3 + 6 + 2 = 11$, agreeing with $(3^3 + 2 \cdot 3)/3 = 11$.

16.13 Integrated checks

17

(a) False. $GL_1(\mathbb{R}) \cong \mathbb{R}^\times$ is abelian. Noncommuting two-by-two matrices and their block extensions establish nonabelianness only for dimensions at least 2.

(b) False. Zero has no multiplicative inverse at any prime modulus. The multiplicative group is $\mathbb{Z}_p^\times$.

(c) False. Representative-independent coset multiplication requires normality, as Problem 6 proves. Problem 5 supplies a concrete nonnormal counterexample.

(d) False. Zero syndrome means that the received word belongs to the code. An error equal to a nonzero codeword can change one valid codeword into another with zero syndrome. For example, changing 000 to 111 in the repetition code remains undetected by its membership test.

(e) False. Burnside divides the fixed-count sum by finite group order $|G|$, not configuration count $|X|$. The fixed-pair argument in Problem 14 identifies the reason.

Transferring group and coding structures

18

Put $t = \varphi([1]_{18})$. The condition $18t \equiv 0 \pmod{12}$ requires and permits exactly the even residues. Every homomorphism is $\varphi_t([a]_{18}) = [at]_{12}$. Representatives differing by 18 have images differing by $18t$, so this condition proves representative independence.

Chapter 16 solutions (continued)

t	Image modulo 12	Kernel modulo 18	Image size
0	$\{0\}$	all of $\mathbb{Z}_{18}$	1
2	$\{0,2,4,6,8,10\}$	$\{0,6,12\}$	6
4	$\{0,4,8\}$	$\{0,3,6,9,12,15\}$	3
6	$\{0,6\}$	$\{0,2,4,6,8,10,12,14,16\}$	2
8	$\{0,4,8\}$	$\{0,3,6,9,12,15\}$	3
10	$\{0,2,4,6,8,10\}$	$\{0,6,12\}$	6

None is onto, since every image has at most six elements. Unrestricted reduction fails because $[0]_{18} = [18]_{18}$, whereas $[0]_{12} \neq [18]_{12} = [6]_{12}$. Choosing least nonnegative representatives defines a function but breaks addition. For example, $[17]_{18} + [1]_{18} = [0]_{18}$ maps to 0, while their chosen images add to $[6]_{12}$.

19

An element (a, b) has order $\operatorname{lcm}(o(a), o(b))$, since both coordinates vanish after multiplication by t exactly when both orders divide t. Every element order thus divides $\operatorname{lcm}(m, n)$. If $\gcd(m, n) > 1$, this lcm is less than mn, so no element generates the whole product.

If $\gcd(m, n) = 1$, then $([1]_m, [1]_n)$ has order mn and generates the product. This also covers $m = 1$ or $n = 1$.

All element orders in $\mathbb{Z}_6 \times \mathbb{Z}_3$ divide 6. This group is not cyclic and cannot be isomorphic to $\mathbb{Z}_{18}$. The generators of the latter are precisely the residues coprime to 18,

$$1, 5, 7, 11, 13, 17.$$

Also $6x = 0 \iff 18 \mid 6x \iff 3 \mid x$, giving $0, 3, 6, 9, 12, 15$. Equal group orders do not imply equal element-order distributions or isomorphism.

20

Appending parity is linear and injective because the first seven coordinates retain the original word. Dimension remains 4 and size remains 16. A nonzero original word has weight at least 3. An odd weight increases by one, while an even weight stays unchanged, so the extended weight is at least 4. Original word 1110000 has weight 3 and extends to weight 4. The extended code is therefore $[8, 4, 4]$.

For error vector $(e_1, \ldots, e_8)$, the checks give

Chapter 16 solutions (continued)

$$s = \sum_{i=1}^{7} e_i H_{\cdot i}, \qquad p = \sum_{i=1}^{8} e_i \pmod 2.$$

Assuming at most two errors, the complete decisions are as follows.

Syndrome s	Parity p	Decision
0	0	No error
nonzero	1	One error among the first seven positions. Find its matching column and flip that bit.
0	1	One error in position 8. Flip it.
nonzero	0	Two errors. Flag the word and do not correct it.

A single error among the first seven positions gives its nonzero column. Position 8 contributes no syndrome. Two errors among the first seven positions give distinct columns whose XOR is nonzero. If one of two errors is in position 8, the syndrome is the other position's column. Two errors always have even parity. These cases exhaust weights 0, 1 and 2.

With three or more errors allowed, neither a no-error decision nor an asserted single-error position is guaranteed. A weight-4 extended codeword used as the error has $s = 0, p = 0$ but changes one valid word into another. Some weight-3 errors produce a single-error interpretation and a miscorrection. The contract is single-error correction and double-error detection, abbreviated SECDED.

21

Shift	Number	Cycle type
0	1	1^6
1 or 5	2	6^1
2 or 4	2	3^2
3	1	2^3

Thus $Z_{C_6} = (x_1^6 + 2x_6 + 2x_3^2 + x_2^3)/6$. Substitute $x_k = r^k + b^k$ and extract $r^3 b^3$. The four terms contribute 20, 0, 4 and 0, so there are $24/6 = 4$ rotational orbits.

The six reflections split into three axes through opposite vertices of type $1^2 2^2$, and three axes through opposite edges of type 2^3. For the former,

$$[r^3 b^3](r + b)^2 (r^2 + b^2)^2 = 4,$$

Chapter 16 solutions (continued)

because the first factor must contribute $2rb$ and the second $2r^2b^2$. The latter permit only even red multiplicities and contribute zero. The full count is

$$\frac{24 + 3 \cdot 4 + 3 \cdot 0}{12} = 3.$$

Dividing 20 by six assumes all orbits have size six. The alternating word has stabilizer shifts 0, 2 and 4, so its orbit has size two. Representatives 000111, 001011, 001101 and 010101 have orbit sizes 6, 6, 6 and 2, totaling 20. Reflection merges the two middle orbits.

22

On the two diagonal sets, e, r^2 act identically and r, r^3 interchange them. There are four binary configurations. Each identity action fixes all four, while an interchange fixes only the two monochromatic configurations. In order e, r, r^2, r^3, the fixed counts are

$$4, 2, 4, 2.$$

Burnside gives $(4 + 2 + 4 + 2)/4 = 3$ orbits, represented by both red, both blue and one of each. A configuration with one of each has stabilizer $\{e, r^2\}$, so its orbit size is $4/2 = 2$.

Burnside's double counting uses the action axioms and orbit-stabilizer, without requiring an injective map into a permutation group. Group elements with the same action still count separately. Alternatively use the effective group $C_4/\{e, r^2\} \cong C_2$, with fixed counts 4 and 2. The average $(4 + 2)/2 = 3$ agrees, provided both numerator and denominator change together.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 16 solutions (continued)

Verification code appendix: verify_burnside_fixed_orbits.py

```
from itertools import product
import json
from pathlib import Path

permutations = [(0,1,2,3),(1,2,3,0),(2,3,0,1),(3,0,1,2)]
words = list(product((0,1), repeat=4))
def act(word, permutation):
    result = [None]*4
    for position, image in enumerate(permutation):
        result[image] = word[position]
    return tuple(result)
def cycles(permutation):
    unseen = set(range(4))
    result = []
    while unseen:
        first = min(unseen)
        cycle, current = [], first
        while current in unseen:
            unseen.remove(current)
            cycle.append(current+1)
            current = permutation[current]
        result.append(cycle)
    return result
fixed = [sum(act(word, p) == word for word in words) for p in permutations]
assert fixed == [16,2,4,2]
assert fixed == [2**len(cycles(p)) for p in permutations]
orbits = {min(act(word,p) for p in permutations) for word in words}
assert len(orbits) == 6 and sum(fixed) == 4*len(orbits)
assert fixed[0] != len(orbits)
assert fixed[1] != len(orbits)
result =
{'fixed_counts':fixed,'fixed_sum':sum(fixed),'group_order':4,'orbit_count':len(orbits),
          'permutations':[{'images':[i+1 for i in p],'cycles':cycles(p),'fixed_binary':f} for
p,f in zip(permutations,fixed)],
          'orbit_representatives':[''.join(map(str,w)) for w in sorted(orbits)],
          'orbit_sizes':[len({act(word,p) for p in permutations}) for word in
sorted(orbits)]}
print(json.dumps(result))
```

Chapter 16 solutions (continued)

Verification code appendix: verify_coding_burnside.py

```
"""Recompute the RSA, Hamming, Burnside, and Polya examples."""

from collections import Counter
from itertools import product


def mat_mul_2x2(
    a: tuple[tuple[int, int], tuple[int, int]],
    b: tuple[tuple[int, int], tuple[int, int]],
) -> tuple[tuple[int, int], tuple[int, int]]:
    return tuple(
        tuple(sum(a[i][k] * b[k][j] for k in range(2)) for j in range(2))
        for i in range(2)
    )


A = ((1, 1), (0, 1))
B = ((1, 0), (1, 1))
assert mat_mul_2x2(A, B) != mat_mul_2x2(B, A)
assert {a for a in range(9) if __import__("math").gcd(a, 9) == 1} == {1, 2, 4, 5, 7, 8}


def compose(p: tuple[int, ...], q: tuple[int, ...]) -> tuple[int, ...]:
    """Return p after q on zero-based positions."""
    return tuple(p[q[i]] for i in range(len(p)))


identity = (0, 1, 2)
transposition = (1, 0, 2)
g = (2, 1, 0)
alternate = compose(g, transposition)
left_coset = {compose(g, h) for h in (identity, transposition)}
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 16 solutions (continued)

```
right_coset = {compose(h, g) for h in (identity, transposition)}
assert left_coset != right_coset
assert compose(g, g) in {identity, transposition}
assert compose(alternate, alternate) not in {identity, transposition}


def mat_vec_mul_mod2(matrix: list[list[int]], vector: tuple[int, ...]) -> tuple[int, ...]:
    return tuple(sum(a * b for a, b in zip(row, vector)) % 2 for row in matrix)


H = [
    [1, 1, 0, 1, 1, 0, 0],
    [1, 0, 1, 1, 0, 1, 0],
    [0, 1, 1, 1, 0, 0, 1],
]
G = [
    [1, 0, 0, 0, 1, 1, 0],
    [0, 1, 0, 0, 1, 0, 1],
    [0, 0, 1, 0, 0, 1, 1],
    [0, 0, 0, 1, 1, 1, 1],
]

assert all(set(mat_vec_mul_mod2([h], tuple(row))) == {0} for h in H for row in G)

codewords = {
    word for word in product((0, 1), repeat=7)
    if mat_vec_mul_mod2(H, word) == (0, 0, 0)
}
assert len(codewords) == 16
assert min(sum(word) for word in codewords if any(word)) == 3
assert (1 + 7) * len(codewords) == 2**7
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 16 solutions (continued)

```
message = (1, 0, 1, 0)
codeword = tuple(
    sum(message[i] * G[i][j] for i in range(4)) % 2
    for j in range(7)
)
assert codeword == (1, 0, 1, 0, 1, 0, 1)
assert mat_vec_mul_mod2(H, codeword) == (0, 0, 0)

received = (1, 1, 1, 0, 1, 0, 1)
syndrome = mat_vec_mul_mod2(H, received)
assert syndrome == (1, 0, 1)
corrected = list(received)
corrected[1] ^= 1
assert tuple(corrected) == codeword

# RSA, including a message that shares a factor with n=55.
assert pow(7, 17, 55) == 17
assert pow(17, 33, 55) == 7
assert pow(15, 17, 55) == 5
assert pow(5, 33, 55) == 15


def necklace_orbit_count(n: int, colors: int) -> int:
    fixed_sum = 0
    for shift in range(n):
        cycles = 0
        seen = [False] * n
        for start in range(n):
            if seen[start]:
                continue
            cycles += 1
            i = start
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 16 solutions (continued)

```
            while not seen[i]:
                seen[i] = True
                i = (i + shift) % n
        fixed_sum += colors**cycles
    return fixed_sum // n


assert necklace_orbit_count(3, 2) == 4
assert necklace_orbit_count(3, 3) == 11
assert necklace_orbit_count(4, 3) == 24

# D4 fixed-coloring table for three colors on four square vertices.
d4_numerator = 81 + 2 * 3 + 9 + 2 * 27 + 2 * 9
assert d4_numerator == 168
assert d4_numerator // 8 == 21


def canonical_necklace(word: tuple[str, ...]) -> tuple[str, ...]:
    return min(word[i:] + word[:i] for i in range(len(word)))


inventory = Counter()
orbits = {canonical_necklace(coloring) for coloring in product("rwb", repeat=3)}
for orbit in orbits:
    inventory[tuple(sorted(Counter(orbit).items()))] += 1

assert inventory[(("b", 1), ("r", 1), ("w", 1))] == 2
assert sum(inventory.values()) == 11

print("RSA, coding, Burnside, and Polya examples: PASS")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study